

A PROJECT REPORT ON

**PREPSMART - AI POWERED MOCK
INTERVIEW AND TEST PLATFORM**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Sarthak Nirgude
Sumit Sunke
Niraj Karande
Rushikesh Mane

Exam No: B400050521
Exam No: B400050604
Exam No: B400050436
Exam No: B400050500



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
**PREPSMART - AI POWERED MOCK INTERVIEW
AND TEST PLATFORM**

Submitted by

Sarthak Nirgude	Exam No: B400050521
Sumit Sunke	Exam No: B400050604
Niraj Karande	Exam No: B400050436
Rushikesh Mane	Exam No: B400050500

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. P. J. Jambhulkar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. P. J. Jambhulkar
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**PREPSMART - AI POWERED MOCK INTERVIEW AND TEST PLATFORM**’.*

*We would like to take this opportunity to thank **Prof. P. J. Jambhulkar** for providing us with all the help and guidance we needed. We are truly grateful for his kind support and valuable suggestions.*

*We are also grateful to **Dr. G. V. Kale**, Head of the Computer Engineering Department, PICT, for her indispensable support and suggestions.*

Sarthak Nirgude
Sumit Sunke
Niraj Karande
Rushikesh Mane
(B.E. Computer Engg.)

ABSTRACT

The project presents a comprehensive and innovative web-based platform aimed at enhancing both academic assessment and interview readiness through adaptive testing and AI-powered analysis. The platform is designed with two core functionalities: an intelligent MCQ testing system and a personalized interview simulation environment. The adaptive MCQ platform dynamically adjusts question difficulty based on the student's real-time performance. Utilizing a secure JWT-based authentication mechanism, the system intelligently fetches topic-wise questions from a relational PostgreSQL database via Prisma ORM, ensuring that previously attempted questions are excluded. This logic-driven test environment is built using Node.js, Express.js, and React.js to provide a seamless and responsive user experience.

Complementing the academic module is a smart interview preparation platform that simulates real-time interview scenarios. It captures user input regarding job role, experience level, and preferences, and uses the Google Gemini API to generate domain-specific questions. Users interact through a webcam-enabled interface, where facial expressions are periodically captured and analyzed using OpenCV and TensorFlow.js to infer emotional cues and confidence levels. Concurrently, audio responses are transcribed using speech-to-text services and analyzed for grammatical correctness, fluency, and relevance using natural language processing techniques. The application follows a modular microservices architecture to ensure scalability, maintainability, and efficient deployment. DevOps practices including Docker containerization and CI/CD pipelines further enhance the reliability of the system. This project bridges the gap between traditional learning tools and intelligent systems, offering a modern solution for improving educational outcomes and professional communication skills. Keywords: Adaptive Assessment, Personalized Learning, AI-Driven Interview Platform, Emotion Detection, Speech Analysis, Google Gemini API, OpenCV, TensorFlow.js, Prisma ORM, React.js, Node.js, Microservices Architecture, JWT Authentication, EdTech, Performance Feedback, DevOps.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem Solving	4
2	Literature Survey	5
2.1	Emotion Detection Using Facial Expressions	6
2.2	Intelligent Multiple Choice Question (MCQ) Testing Systems .	6
2.3	Automated Evaluation and Feedback Mechanisms	7
2.4	Role-Based Authentication and Routing in Scalable Systems .	7
2.5	Real-Time Web Technologies and Multi-Modal Assessment . .	8
2.6	Role of JWT and Role-Based Routing in Scalable Architectures	8
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	11
3.2.1	System Feature 1: Adaptive Question Recommendation	11
3.2.2	System Feature 2: Test State Management and Progression	11
3.2.3	System Feature 3: Result Generation and Performance Analysis	11
3.2.4	System Feature 4: Authentication and Role-Based Access Control	11
3.3	External Interface Requirements	11
3.3.1	User Interfaces	11
3.3.2	Hardware Interfaces	12
3.3.3	Software Interfaces	12
3.3.4	Communication Interfaces	12
3.4	Nonfunctional Requirements	13

3.4.1	Performance Requirements	13
3.4.2	Safety Requirements	13
3.4.3	Security Requirements	13
3.4.4	Software Quality Attributes	13
3.5	System Requirements	14
3.5.1	Database Requirements	14
3.5.2	Software Requirements (Platform Choice)	14
3.5.3	Hardware Requirements	15
3.6	Analysis Models: SDLC Model to be Applied	15
4	System Design	16
4.1	System Architecture	17
4.2	Data Flow Diagram	18
4.3	Use Case Diagram	19
4.4	Sequence Diagram	20
4.5	Activity Diagram	21
4.6	Deployment Diagram	23
5	Project Plan	24
5.1	Project Estimate	25
5.1.1	Reconciled Estimates	25
5.1.2	Project Resources	25
5.2	Risk Management	26
5.2.1	Risk Identification	26
5.2.2	Risk Analysis	26
5.2.3	Overview of Risk Mitigation, Monitoring, Management	26
5.3	Project Schedule	27
5.3.1	Project Task Set	27
5.3.2	Task Network	27
5.4	Team Organization	27
5.4.1	Team structure	27
5.4.2	Timeline Chart	28
5.4.3	Management reporting and communication	29
6	Project Implementation	30
6.1	Overview of Project Modules	31
6.1.1	User Interaction and Input Module	31
6.1.2	Question Generation Module	31
6.1.3	Interview Recording and Media Capture Module	31
6.1.4	Facial Expression Analysis Module	32
6.1.5	Audio Grammar Analysis Module	32

6.1.6	Feedback and Scoring Module	32
6.1.7	Data Storage and Management Module	33
6.1.8	Deployment and DevOps Module	33
6.2	Tools and Technologies Used	33
6.3	Methodology and Algorithm Details	35
6.3.1	AI-Based Question Generation System	35
6.3.2	Facial and Audio Feedback Analysis System	35
6.3.3	User Management and Microservices Architecture	36
6.3.4	Algorithm: AI Interview Preparation Workflow	38
7	Software Testing	39
7.1	Test Cases & Test Results	40
7.2	Load Testing Results	40
8	Results	42
8.1	Results	43
8.1.1	Home Page	43
8.1.2	Interview Setup Form	44
8.1.3	Interview Page	45
8.1.4	AI Feedback Page	46
8.1.5	System Effectiveness	47
9	Conclusions	48
9.1	Conclusions	49
9.2	Future Work	49
9.3	Applications	50
10	Appendix	51
10.1	Appendix A	52
10.2	Appendix B	53
10.2.1	Survey Paper Details:	53
10.2.2	Implementation Paper Details:	53
10.3	Appendix C	54
11	References	55

List of Figures

4.1	System Architecture Diagram	17
4.2	Data Flow Diagram	18
4.3	Use Case Diagram	19
4.4	Sequence Diagram	20
4.5	Deployment Diagram	22
4.6	Deployment Diagram	23
6.1	Microservice Architecture Diagram	37
8.1	PrepSmart Home Page with AI-Powered Interview Preparation Features and Get Started Call-to-Action	43
8.2	Commonly asked questions	44
8.3	Interview Setup Form with Job Description, Role, Experience Level, and Question Count	45
8.4	Live Interview Interface with Camera and Audio Input	46
8.5	AI-Powered Feedback after Interview Session	47
8.6	Question with their correct answers and recorded answers	47
10.1	Plagiarism Scan Report A	54
10.2	Plagiarism Scan Report B	54

CHAPTER 1

INTRODUCTION

1.1 Overview

In today's AI-driven applications, understanding user emotions is critical for enhancing user experience, especially in contexts like virtual interviews or interactive platforms. Our project addresses this by developing a real-time emotion detection system that leverages facial expressions to generate meaningful feedback. Built using a Convolutional Neural Network (CNN) with TensorFlow/Keras, the system processes webcam-captured images to classify emotions accurately. A backend module interprets the emotional sequence to provide personalized feedback or summaries, improving self-awareness and communication. Additionally, the system supports optional data storage and integrates seamlessly into live platforms, making it a powerful tool for emotion-aware human-computer interaction.

1.2 Motivation

In today's rapidly evolving educational and professional landscape, traditional learning and assessment methods often fall short of preparing individuals for dynamic and competitive environments. Students are commonly evaluated through static assessments that do not adapt to their understanding level, while interview preparation remains informal, lacking structured feedback on communication and confidence. This project is driven by the need to bridge these gaps through a smart, integrated platform that combines adaptive academic assessment with AI-powered interview simulation. The motivation stems from the increasing demand for personalized learning and realistic interview practice tools that can analyze not just technical responses but also verbal and non-verbal cues. By leveraging modern web technologies, machine learning, and large language models, the platform aims to enhance both academic performance and professional readiness among students and job seekers.

1.3 Problem Definition and Objectives

Problem

To design and develop a web-based platform that provides dynamic MCQ testing by adapting question difficulty based on student performance. also simulate real-time interviews using AI-generated questions and analyze verbal and non-verbal responses for intelligent feedback and integrate

modern web technologies and machine learning techniques for personalized assessment and professional readiness.

Objectives

- Develop an emotion detection system using facial expressions by training a custom CNN model on a labeled image dataset.
- Integrate the emotion detection model into a web interface to capture real-time images and provide emotional feedback to users.
- Enable automated feedback generation based on the detected emotional state of the user.
- Design and implement a dynamic MCQ testing module where the flow of questions adapts based on the user's performance in previous questions.
- Maintain a backend logic that adjusts difficulty levels or selects next questions in real-time, based on correctness and speed of previous answers.
- Provide students with insightful feedback and performance analytics after test completion to support personalized learning.
- Ensure a smooth user experience across both modules with a centralized, easy-to-use web platform.

1.4 Project Scope & Limitations

Scope

- Detect user emotions in real-time using facial expressions captured through a web interface.
- Train and deploy a Convolutional Neural Network (CNN) model to accurately classify basic emotions.
- Provide automated feedback to users based on detected emotions for applications such as mental wellness and adaptive learning.
- Implement a dynamic MCQ testing system that selects the next question based on user responses.

- Adjust question difficulty levels in real-time to match the learner's understanding and performance.
- Track and analyze user performance to generate personalized results and insights.

Limitations

- **Emotion Detection Accuracy:** Performance may vary based on lighting conditions, camera quality, and facial visibility.
- **Limited Emotion Categories:** The model detects only a predefined set of emotions (e.g., happy, sad, angry, etc.).
- **Response Dependency:** In the MCQ module, test flow is heavily dependent on the accuracy of previous answers, which may create bias in performance evaluation.
- **Scalability Constraints:** Handling large numbers of concurrent users may require significant server resources and optimization.
- **Webcam Access Requirement:** Emotion detection functionality is restricted to devices with camera access and user permission.

1.5 Methodologies of Problem Solving

- **Emotion Detection:** A webcam interface captures live facial expressions, which are preprocessed and fed into a trained Convolutional Neural Network (CNN) model to classify user emotions.
- **Model Training:** The CNN is trained using labeled datasets consisting of various facial expressions categorized into basic emotions such as happy, sad, angry, surprised, etc.
- **Feedback Generation:** Based on the detected emotion, appropriate feedback or suggestions are displayed to the user to enhance engagement or well-being.
- **Dynamic MCQ Logic:** A logic engine dynamically selects the next question based on the user's response to the current one. Correct answers may lead to more challenging questions, while incorrect responses lead to simpler or conceptually related ones.

CHAPTER 2

LITERATURE SURVEY

The evolution of educational technology has seen a rise in adaptive learning platforms and intelligent tutoring systems. Simultaneously, artificial intelligence has found widespread use in personalizing user experiences, including in areas such as interview training and communication assessment. This chapter reviews significant works that support the theoretical and practical foundation of the proposed platform, which combines adaptive MCQ testing and AI-powered interview simulation.

2.1 Emotion Detection Using Facial Expressions

Emotion recognition using facial expressions has gained substantial traction in educational technology, mental health diagnostics, and interactive systems. Initial efforts in this domain utilized handcrafted feature extraction methods involving geometric facial landmarks and statistical classifiers like Support Vector Machines (SVM) and k-NN. However, these approaches suffered in uncontrolled environments due to lighting variation and occlusions. The emergence of deep learning, particularly Convolutional Neural Networks (CNNs), significantly improved robustness and accuracy. Datasets such as FER-2013, CK+, and AffectNet have become benchmarks for training and evaluation of emotion recognition systems. Studies like Kanagaraju et al. (2022) utilized the VGG16 architecture, demonstrating over 70 accuracy in real-time emotion inference using FER-2013. Similarly, transfer learning with MobileNet-V2 and Inception-V3 has shown promising results, especially on nuanced datasets like Emognition, which includes rare emotions such as awe and enthusiasm. Despite progress, challenges persist in generalizing across ethnicities, lighting conditions, and micro-expressions. Our project addresses these gaps by implementing a real-time CNN-based system integrated into a web application, enabling seamless emotion analysis during live interview simulations.

2.2 Intelligent Multiple Choice Question (MCQ) Testing Systems

MCQ-based assessments have long been a cornerstone in digital learning platforms due to their scalability and objectivity. Traditional systems offer static question sets with linear progression, limiting adaptivity. Contemporary research focuses on intelligent tutoring systems and adaptive

testing engines that modify question difficulty based on learner performance. For instance, platforms like Duolingo and Knewton dynamically curate question sequences using decision trees or Bayesian Knowledge Tracing (BKT). Studies show that adaptive MCQ engines increase engagement and retention by 30-40 percent (Zhou et al., 2020). Algorithms based on Item Response Theory (IRT) have further enhanced these systems by modeling both question difficulty and learner ability. Our platform incorporates an adaptive flow mechanism that modifies question levels based on real-time response analysis, thus creating a personalized test environment suitable for preparatory and diagnostic purposes.

2.3 Automated Evaluation and Feedback Mechanisms

The automation of test evaluation has transitioned from simple answer-matching to more intelligent feedback systems that incorporate time analysis, option selection patterns, and contextual scoring. Modern systems use predefined logic trees and rule-based engines to assess performance, followed by structured feedback generation. Prominent platforms such as HackerRank and Edmodo utilize PDF generators (like PDFKit) and messaging services (SMTP or Nodemailer) to deliver timely reports. Integration of Natural Language Generation (NLG) tools for feedback narration has emerged as a research trend, with frameworks like GPT-powered summarizers being trialed for academic assessment commentary. In our work, we leverage PDFKit for generating performance reports and Nodemailer for dispatching these to mentors and users. Planned enhancements include using LLMs for generating narrative feedback summaries based on user scores and emotion logs.

2.4 Role-Based Authentication and Routing in Scalable Systems

Large-scale educational platforms require clear user segregation to maintain security and user experience quality. Role-based access control (RBAC) is widely adopted in platforms like Moodle, Coursera, and Microsoft Learn. These systems use JSON Web Tokens (JWTs) and role-specific routing to isolate functionalities and interfaces. Research has emphasized the importance of token-based stateless authentication to scale services

without compromising performance or session integrity. Our system employs JWT-based authentication to ensure secure login sessions and uses middleware functions for role-based routing. Students access endpoints like `/start-test`, `/submit-response`, and `/emotion-track`, whereas mentors and evaluators are routed to `/upload-questions`, `/view-reports`, and `/session-summary`. This separation enhances code modularity, security, and deployment scalability.

2.5 Real-Time Web Technologies and Multi-Modal Assessment

Recent research explores real-time systems that integrate webcam, audio, and behavioral data during assessments to evaluate not just cognitive ability but also emotional and psychological states. Platforms like Mettl and ProctorU use AI-enabled proctoring to track gaze, facial emotion, and even audio cues during online tests. Multi-modal systems combine affective computing (emotion detection), response correctness, and timing data to provide a holistic view of a learner's performance. Our approach integrates real-time facial emotion analysis with adaptive MCQ testing, allowing for emotion-aware feedback. This multi-dimensional framework not only enhances test validity but also provides actionable insights for mentors regarding student stress levels, focus patterns, and overall confidence.

2.6 Role of JWT and Role-Based Routing in Scalable Architectures

Secure authentication and modular user experience are essential in platforms serving multiple stakeholders. PREPSMART employs JSON Web Tokens (JWT) for secure and stateless authentication. This ensures session integrity across distributed systems while reducing backend load. Moreover, the system supports role-based routing, ensuring that students, educators, and platform administrators receive tailored interfaces and functionalities. For instance, students access routes like `/start-test` and `/submit-response`, while vendors and educators navigate through `/viewPerformanceReport` and `/uploadQuestions`. This separation of concerns improves maintainability, code organization, and scalability—critical for production-grade learning platforms.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

This section outlines the key assumptions made during the development of the PlacementPrep system and the external dependencies that are essential for its successful operation and deployment:

- The platform assumes a stable internet connection for both students and administrators to ensure seamless communication with the backend services.
- Students are expected to have access to a modern web browser that supports JavaScript, HTML5, and CSS3 for full compatibility with the frontend interface.
- The recommendation logic is dependent on the accuracy of question tagging based on difficulty levels and topic categorization in the database.
- The backend assumes consistent structure and availability of question data, answer options, and correctness flags within the database schema.
- The recommendation system assumes that students will answer questions in a linear flow and not skip questions arbitrarily during a session.
- The platform depends on JSON Web Tokens (JWT) for secure user authentication and authorization.
- The system uses Prisma ORM and a PostgreSQL database; any changes to database structure or configuration should be properly synchronized using migrations.
- For scalability and performance, the platform assumes that the server is deployed in a production-ready environment with sufficient memory and CPU resources.
- The frontend assumes React state will be managed reliably across components, particularly for tracking test progress, user performance, and recommended questions.

3.2 Functional Requirements

3.2.1 System Feature 1: Adaptive Question Recommendation

The system will present questions to students based on their performance using an adaptive logic. Initial questions are served from level 1, and subsequent questions are selected based on correctness and difficulty progression using a predefined algorithm.

3.2.2 System Feature 2: Test State Management and Progression

Student progress will be managed on the frontend using persistent state logic. This includes tracking the current level, question index, total questions attempted, and dynamically determining the next question without refreshing or restarting the session.

3.2.3 System Feature 3: Result Generation and Performance Analysis

After test completion, the system will calculate and display a summary of performance metrics including score, time taken, and strength-weakness analysis based on topic-wise and level-wise performance. Visualizations will be powered by Chart.js.

3.2.4 System Feature 4: Authentication and Role-Based Access Control

User authentication will be handled using JWT. The system will support multiple roles including students and administrators. Role-based access will restrict certain features (e.g., question uploads, data analysis) to admins only.

3.3 External Interface Requirements

3.3.1 User Interfaces

The user interacts with the platform through a web-based interface built using React, hosted on Vercel. The interface includes features such as:

- Test selection and initiation
- Real-time question and answer display
- Adaptive question recommendation based on student performance
- Test results and performance analytics
- Admin dashboard for content and user management

3.3.2 Hardware Interfaces

- Frontend hosted on a cloud service (Vercel) for accessibility across devices.
- Backend server (Node.js with Express) for orchestrating business logic and handling test data.
- Database server (PostgreSQL managed with Prisma ORM) for storing test content, user data, and performance metrics.
- Optional: Cloud storage or CDN for serving static assets and reports.

3.3.3 Software Interfaces

1. Operating System: Linux-based OS (Ubuntu or similar preferred)
2. Backend: Node.js (Express) for API and business logic orchestration
3. Database: PostgreSQL for data storage, Prisma ORM for interaction
4. Frontend: React for user interface and interaction
5. Authentication: JWT-based authentication for secure login
6. Email: Nodemailer/Brevo API for sending test results and notifications

3.3.4 Communication Interfaces

- RESTful APIs between React frontend, Node.js backend, and PostgreSQL database.
- WebSocket or polling for live test progression and adaptive question updates.
- Email notifications for test results and updates sent via Nodemailer/Brevo.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

The system should ensure smooth performance under load:

- Test questions should load within 2 seconds after initiation.
- Student progress (including real-time level progression) should update within 1 second.
- Test results and performance analytics should be computed and displayed within 3 seconds.

3.4.2 Safety Requirements

- The system must handle network or server failures gracefully, ensuring no data loss during test sessions.
- In the event of failure, students should be able to resume tests from the last valid state.
- Backups should be performed regularly to prevent loss of test data and user progress.

3.4.3 Security Requirements

- All interactions (test initiation, result submission) must be authenticated using JWT tokens to ensure secure access.
- API communications must be secured using HTTPS to protect user data.
- User passwords and sensitive data must be encrypted at rest.
- Role-based access control (RBAC) for admins and students to restrict unauthorized access to content management features.

3.4.4 Software Quality Attributes

- **Scalability:** The system must scale to support a growing number of users without performance degradation.

- **Reliability:** The system must be highly available, with minimal downtime for maintenance and upgrades.
- **Usability:** The user interface should be intuitive, providing a seamless experience for students and admins.
- **Maintainability:** The codebase should follow modular and component-based design patterns to facilitate easy updates and bug fixes.

3.5 System Requirements

3.5.1 Database Requirements

- PostgreSQL database for storing test data, user profiles, performance metrics, and question progression.
- Prisma ORM for efficient data manipulation and interaction between the backend and database.
- Redis or in-memory cache for storing session data and temporary user progress to ensure fast retrieval during tests.
- Cloud storage (optional) for static assets such as PDFs, reports, or large test data files.

3.5.2 Software Requirements (Platform Choice)

- Node.js backend with Express deployed on Render for orchestration and handling business logic.
- React frontend deployed on Vercel for interactive user experience and responsive test interfaces.
- Prisma ORM for handling database operations in the backend.
- JWT-based authentication for secure access to the platform.
- GitHub Actions for CI/CD pipeline automation, ensuring smooth deployment and version control.

3.5.3 Hardware Requirements

- Application server with 4-8 GB RAM and a 2-core CPU for handling backend operations and test-related business logic.
- PostgreSQL database server with sufficient disk space (SSD preferred for fast data access) to store test content, user data, and session information.
- Vercel hosting for frontend deployment with auto-scaling based on user traffic.
- Cloud infrastructure for supporting scaling, particularly during peak traffic periods (e.g., AWS, Google Cloud).

3.6 Analysis Models: SDLC Model to be Applied

The Agile methodology will be used to guide the development process. The project will be divided into multiple iterative sprints, allowing for frequent releases and user feedback. Key points of the SDLC model include:

- **Sprint Planning:** At the beginning of each sprint, key features and goals for the sprint will be outlined.
- **Iteration:** Development will be done in short iterations to ensure quick delivery of working components, such as test module functionalities or user performance tracking.
- **Testing:** Continuous testing will be performed for each module to ensure correctness and quality.
- **Feedback Loops:** At the end of each sprint, a demo will be held with the stakeholders (admins, students) to gather feedback on the features delivered.
- **Continuous Integration/Continuous Deployment (CI/CD):** Automated testing and deployment pipelines using GitHub Actions will be set up to ensure that new features are tested and deployed seamlessly.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture of PrepSmart is based on a robust microservices architecture, designed to ensure high scalability, modular and maintainable architecture, seamless integration between services, and optimal performance under varying loads. The frontend is developed using Next.js and React, providing users with an intuitive interface to simulate mock interviews. The backend, built with Node.js and Express, manages API requests, end-user authentication, and inter-service communication. It integrates with Gemini APIs for dynamic question generation based on user input such as job role and experience. Media data like webcam images and audio responses is processed by dedicated facial expression analysis and audio grammar checking services. These services return feedback on user confidence and response quality, which is aggregated and displayed via the frontend. All data is securely stored in a PostgreSQL database hosted on Neon, and accessed using Drizzle ORM for type-safe database operations. The system uses JWT-based authentication and is deployed via Vercel and Render, with support for Dockerized services to maintain isolated, scalable environments.

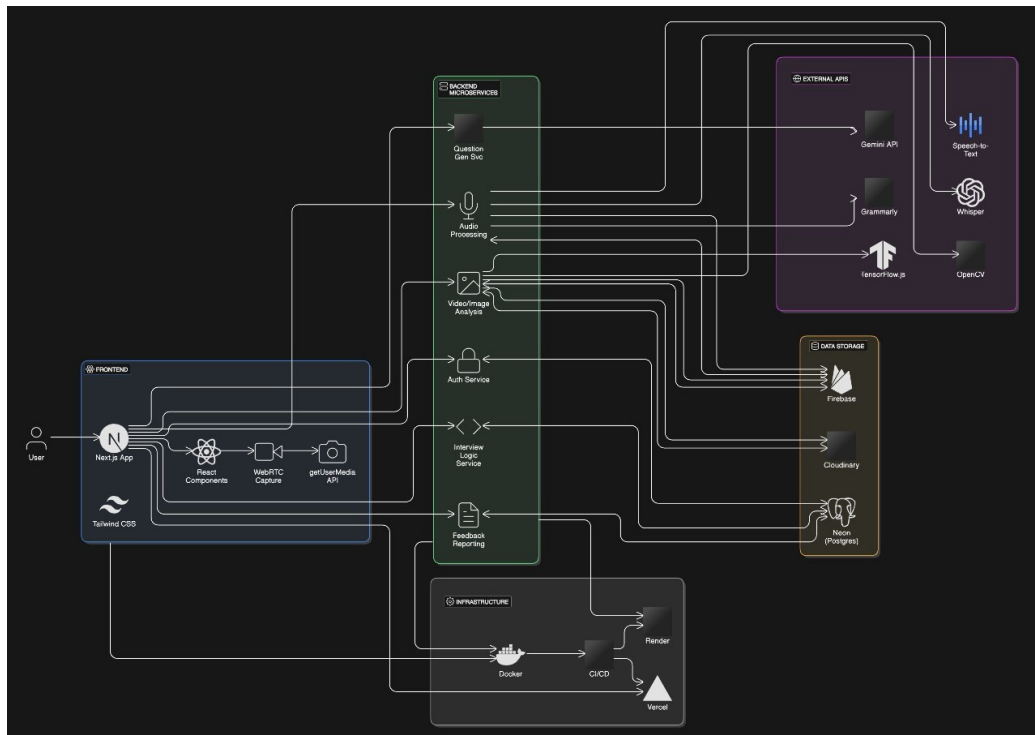


Figure 4.1: System Architecture Diagram

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) of PrepSmart represents the logical flow of data within the system and outlines how information moves between different components. It begins with the user providing input details such as job role, job description, experience level, and the number of questions. This data is processed by the backend server, which communicates with the Gemini API to generate relevant interview questions. During the interview session, audio and webcam data are captured and sent to dedicated microservices for facial expression analysis and grammar checking, respectively.

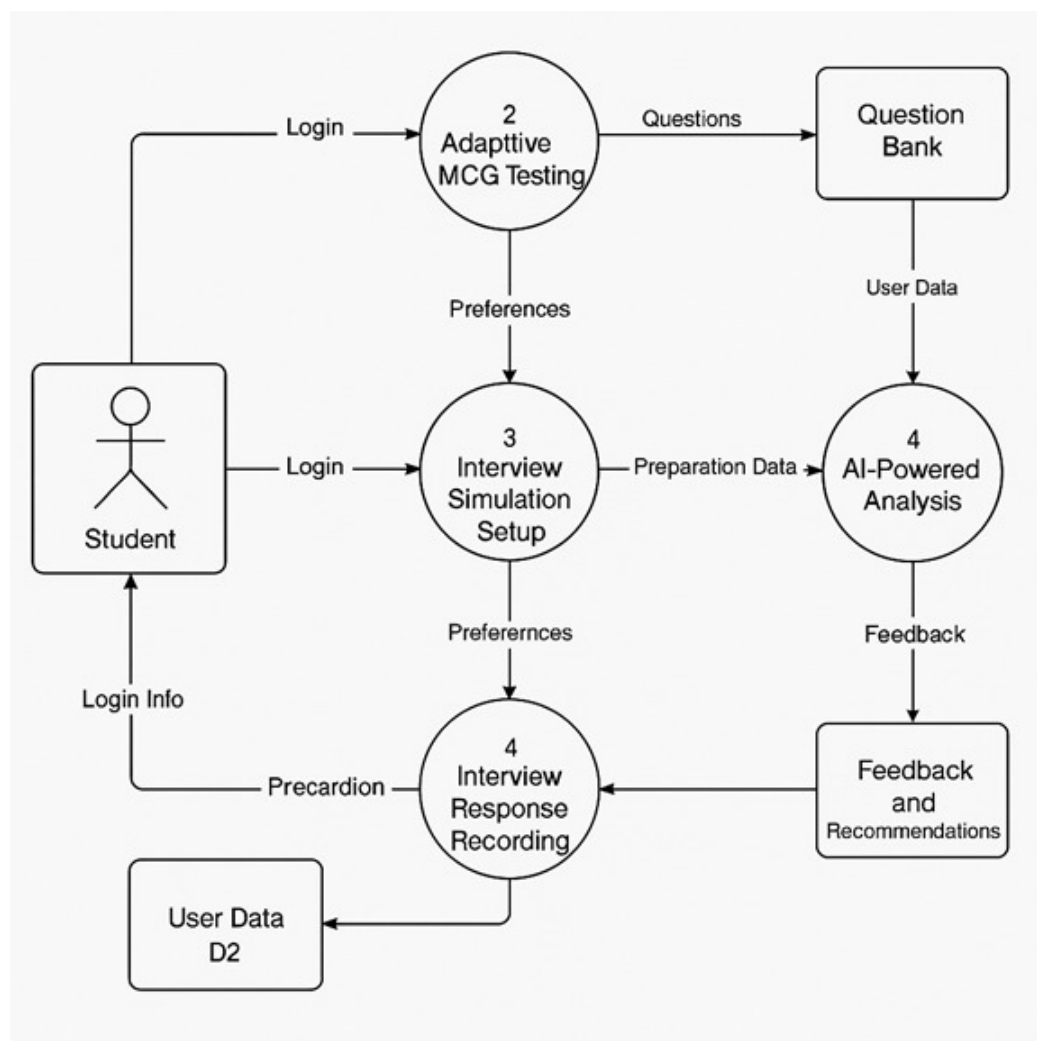


Figure 4.2: Data Flow Diagram

These services return insights such as confidence levels and language accuracy, which are compiled into feedback. The final output is stored in the database and presented to the user through the frontend interface.

4.3 Use Case Diagram

The Use Case Diagram of PrepSmart illustrates the key functionalities of the system from the perspective of the end user. It identifies the primary actor—the User—and outlines their interactions with the system through various use cases.

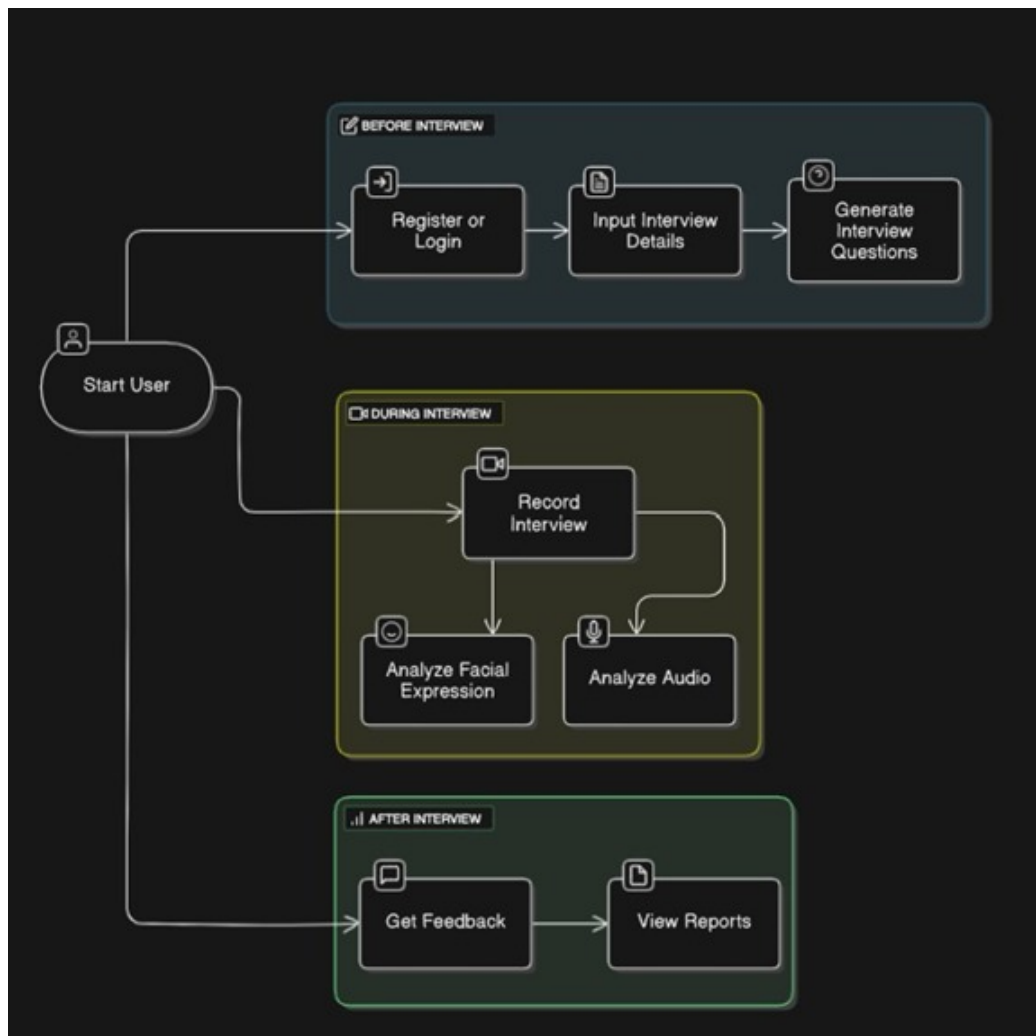


Figure 4.3: Use Case Diagram

These include registering or logging into the platform, entering interview-related details, generating questions using the Gemini API, starting a mock interview session, recording audio and video responses, and receiving AI-powered feedback. Additional use cases include analyzing facial expressions and spoken responses, viewing performance reports, and receiving improvement suggestions. This diagram helps in capturing the system's functional requirements and provides a high-level overview of how users interact with different features of the platform.

4.4 Sequence Diagram

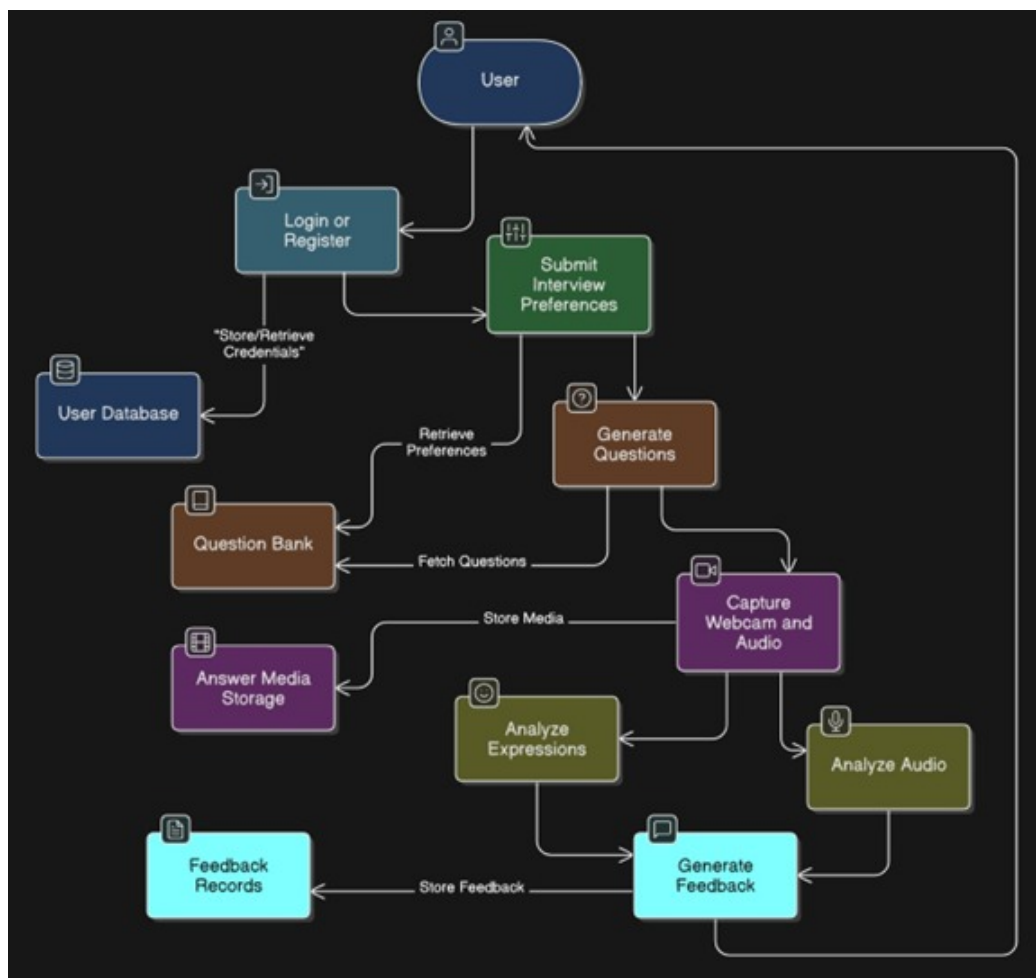


Figure 4.4: Sequence Diagram

The Sequence Diagram of PrepSmart models the dynamic interaction

between various system components during a typical interview session. It showcases the chronological sequence of messages exchanged between the user, frontend interface, backend server, and multiple microservices such as the Gemini API, facial expression analysis service, and audio analysis service. The interaction begins when the user logs in and submits their interview preferences. The backend then requests and receives generated questions from the Gemini API. As the user records their responses, image and audio data are sent to their respective analysis services. These services process the inputs and return results, which the backend compiles into personalized feedback. The diagram effectively illustrates the flow of control, request-response cycles, and real-time data processing, helping to visualize system behavior over time.

4.5 Activity Diagram

The Activity Diagram of PrepSmart presents a comprehensive visualization of the user's journey through the platform and highlights the system's internal processing at each stage. The workflow initiates when a user accesses the application and either logs in or registers using secure authentication. After successful login, the user inputs their interview preparation preferences, including job role, job description, experience level, and the desired number of questions. The system then communicates with the Gemini API to dynamically generate tailored interview questions. Upon initiating the mock interview, the user begins answering questions while the platform captures both webcam images and audio responses simultaneously. These data streams are sent to independent microservices for facial expression analysis—which determines emotional state and confidence level—and audio analysis, which checks for grammatical accuracy and coherence. After the processing is complete, the platform compiles all results to generate comprehensive feedback along with improvement suggestions. The user can then view their performance report, which includes ratings, tips, and a summary of strengths and weaknesses. The activity diagram effectively outlines this end-to-end process, including decision points (e.g., validation of inputs, media capture success), parallel actions, and possible exception flows, offering a clear and structured understanding of how users interact with the system in real-time.

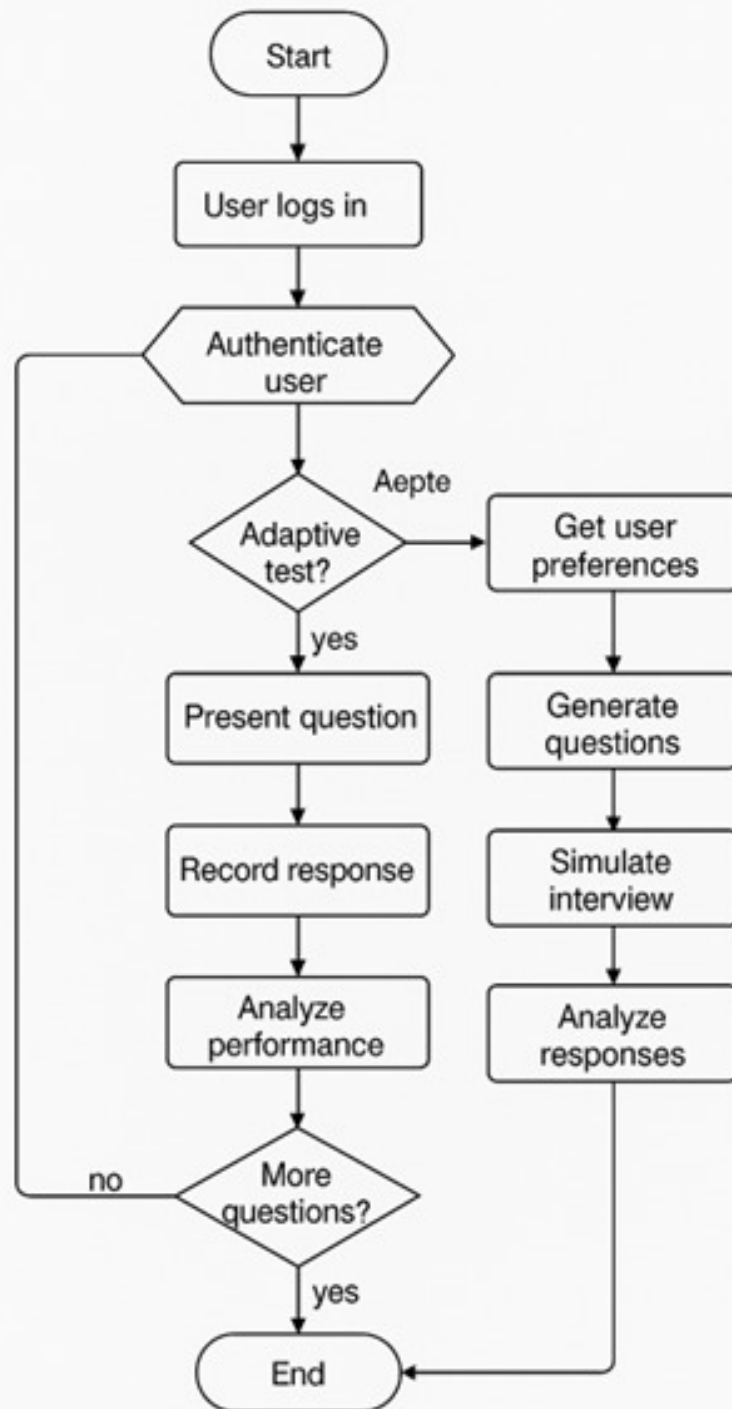


Figure 4.5: Deployment Diagram

4.6 Deployment Diagram

The Deployment Diagram of PrepSmart illustrates the physical architecture of the system, highlighting how various software components are deployed across different nodes and environments. The frontend application, developed using Next.js and React, is deployed on Vercel, providing a fast and scalable user interface. The backend, built with Node.js and Express, runs on Render and serves as the central processing unit for user requests, business logic, and service orchestration. Two independent Dockerized microservices—one for facial expression analysis and another for audio grammar checking—handle real-time media processing and are managed using Docker Compose. The platform integrates with external services like Gemini API for question generation and uses a PostgreSQL database hosted on Neon, accessed via Drizzle ORM for efficient data operations. Authentication and authorization are managed by a secure Auth service. All components communicate over secure RESTful APIs, and the architecture supports CI/CD pipelines for seamless deployment and updates. The deployment diagram provides a clear view of how system components are distributed and interact in a cloud-based environment.

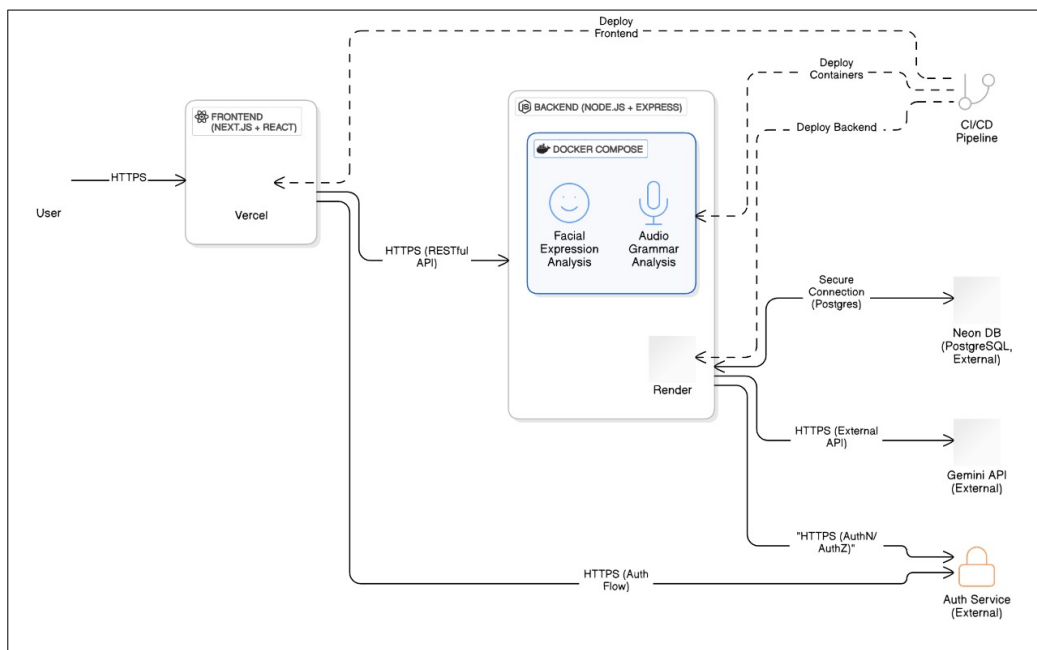


Figure 4.6: Deployment Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

This section outlines the estimated effort and resources required for the successful development of the Adaptive MCQ Test Platform.

5.1.1 Reconciled Estimates

- **Development Time:** Approximately 10–12 weeks, covering requirements gathering, system design, backend/frontend development, testing, and deployment.
- **Team Size:** 4 students with dedicated roles (frontend development, backend/API logic, database design, test logic and analytics).
- **Technology Stack:** React.js, Node.js, Express.js, MongoDB/Prisma, Chart.js, PDFKit, Firebase (for storage), and JWT for authentication.
- **Estimated Cost:** Majorly open-source tools used. Minor expenses for cloud services (Vercel, MongoDB Atlas), and third-party APIs if applicable.

5.1.2 Project Resources

Resources required for the development include:

- Laptops/desktops with at least 8GB RAM for local development and testing
- Hosting platforms (e.g., Vercel for frontend, Render/Render-free tier for backend)
- Database access through MongoDB Atlas or local MongoDB instance
- Access to Brevo (Sendinblue) for automated result/report emails

5.2 Risk Management

To ensure on-time delivery and robust performance, risks are identified and mitigated in advance.

5.2.1 Risk Identification

- Difficulty in managing dynamic test states (tracking level and question index)
- Loss of test progress due to page reloads or session timeout
- Data inconsistency due to improperly structured schemas
- Visualization or report generation errors (e.g., chart or PDF issues)

5.2.2 Risk Analysis

- **High Impact:** Test logic failure or session loss mid-test
- **Medium Impact:** Incorrect chart generation, PDF alignment issues
- **Low Impact:** Minor UI inconsistencies, slow frontend loading

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- Maintain frontend test state using hooks and local/session storage
- Use modular, well-tested APIs and ensure frontend-backend integration consistency
- Validate database schema and structure relationships via Prisma
- Test email and PDF/chart generation in sandbox/staging before production deployment

5.3 Project Schedule

5.3.1 Project Task Set

- Requirement Analysis and UI/UX Wireframe Design
- Backend Architecture Setup and Database Schema Finalization
- Test Engine Implementation (Dynamic question delivery and tracking)
- Analytics, Charts, and PDF Generation Module
- Email Automation, Final Integration, and Deployment

5.3.2 Task Network

- Question recommendation logic must be integrated before chart/PDF modules
- Backend must expose test APIs before frontend MCQ interface is fully functional
- Final deployment depends on completion of testing, integration, and report modules

5.4 Team Organization

5.4.1 Team structure

- Sarthak Nirgude: Responsive design of the MCQ interface and result dashboard
- Sumit Sunke: Backend development including Express.js routes and Prisma integration
- Niraj Karande: State management and session persistence in frontend test engine
- Rushikesh Mane: Development and integration of the emotion detection module

5.4.2 Timeline Chart

Phase	Tasks	Duration
Phase 1: Planning and Requirements Gathering	- Identify user roles (student, admin) and system objectives. - Define behavior-based test flow and emotion feedback use cases.	2 weeks
Phase 2: Design and Architecture	- Design schema for users, tests, questions, and emotion logs. - Plan test engine logic to adapt based on user answers. - Architect integration between frontend, backend, and LLM/ML modules.	3 weeks
Phase 3: Development	3.1 Dynamic Test Logic - Build API endpoints for test flow, user progress, and scoring. 3.2 Emotion Detection Module - Integrate facial emotion detection using OpenCV/Mediapipe. 3.3 Report and Dashboard - Develop performance dashboard with graphs and emotion overlays.	6 weeks
Phase 4: Testing and QA	- Unit test emotion detection and result interpretation logic. - Conduct mock tests to validate progression and LLM accuracy. - Perform UI/UX testing for dynamic MCQ flow.	3 weeks
Phase 5: Deployment	- Deploy frontend (Vercel) and backend (Render). - Set up cron jobs for daily test summaries and emotion trends.	2 weeks
Phase 6: Monitoring and Maintenance	- Monitor API performance and emotion classification accuracy. - Improve LLM-generated feedback with more training data.	Ongoing

Table 5.1: Project Timeline and Task Breakdown

5.4.3 Management reporting and communication

- Weekly progress meetings with project guide
- Updates shared via Google Docs and GitHub
- Communication over WhatsApp and Google Meet for day-to-day coordination

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The **PrepSmart** platform is an AI-powered interview preparation system designed to simulate real-time interviews and provide actionable feedback based on facial expressions and spoken responses. Built with a microservices architecture, it ensures scalability, modularity, and efficient processing across all components.

6.1.1 User Interaction and Input Module

This module collects essential information from users to personalize their mock interview experience.

- Users input data such as **job role**, **job description**, **experience level**, and the **desired number of questions**.
- A secure **authentication service** ensures that users can register and log in safely, utilizing industry-standard authorization mechanisms.
- The UI, built with **Next.js and React**, offers a responsive and intuitive interface optimized for both desktop and mobile.

6.1.2 Question Generation Module

This module dynamically generates personalized interview questions and answers.

- The backend calls the **Gemini API** using user-provided input to fetch relevant interview questions.
- A RESTful microservice processes and formats the API response before sending it to the frontend.
- The system can also provide suggested answers and contextual hints for each generated question.

6.1.3 Interview Recording and Media Capture Module

This module facilitates live recording of interview responses via webcam and microphone.

- Users can record their answers using an embedded media recording feature with dedicated buttons for video and audio capture.

- Webcam images are captured at intervals to analyze facial expressions.
- All media files are securely transmitted to their respective analysis services using HTTPS.

6.1.4 Facial Expression Analysis Module

A dedicated microservice analyzes captured images to assess user confidence based on facial cues.

- Uses computer vision techniques to classify facial expressions as **happy**, **sad**, **neutral**, or **confused**.
- Generates a **confidence rating** for each response and sends it back to the feedback engine.
- Dockerized for isolated deployment and managed via Docker Compose.

6.1.5 Audio Grammar Analysis Module

This microservice processes recorded audio responses to assess clarity, grammar, and fluency.

- Speech is transcribed and analyzed using natural language processing.
- Checks for grammar accuracy, sentence structure, and filler words.
- Provides textual feedback and improvement tips, contributing to overall scoring.

6.1.6 Feedback and Scoring Module

This module compiles analysis results and generates user-centric performance feedback.

- Combines data from facial and audio analysis services to calculate a **composite performance score**.
- Highlights strengths, areas for improvement, and provides **targeted suggestions** for better answers.
- Displays a summary report to the user, along with visual indicators and charts.

6.1.7 Data Storage and Management Module

Responsible for securely storing user data, interview sessions, and feedback logs.

- Uses **Neon** (PostgreSQL as a service) as the primary database.
- Integrated with **Drizzle ORM** for type-safe, schema-aware data operations.
- Ensures proper indexing and query optimization for efficient data retrieval.

6.1.8 Deployment and DevOps Module

Manages deployment pipelines and service orchestration for the PrepSmart platform.

- Frontend is deployed on **Vercel**, and backend services on **Render**.
- Microservices are containerized using **Docker** and orchestrated with Docker Compose.
- CI/CD pipelines ensure seamless updates and rollback capabilities.

6.2 Tools and Technologies Used

The implementation of the **PrepSmart** platform leverages a modern and scalable technology stack, encompassing front-end frameworks, backend services, AI integrations, and cloud infrastructure. These tools work together to provide an interactive, intelligent, and secure interview preparation experience for users. Below is a detailed description of the core tools and their respective roles within the system.

- **Next.js:**
Next.js is used as the core frontend framework to build a fast and optimized React-based single-page application. It provides built-in routing, server-side rendering, and static site generation features, enhancing performance and SEO.
- **React:**
React powers the dynamic UI components, including user input forms, webcam integration, and feedback reports. It allows modular, reusable components and state management for a seamless interview flow.

- **Tailwind CSS:**
Tailwind CSS is employed for rapid and consistent UI styling. Its utility-first approach allows developers to design responsive, accessible, and clean interfaces without writing custom CSS.
- **Node.js:**
Node.js serves as the backend runtime environment to build scalable microservices. It handles API routes, user sessions, request validation, and integration with third-party services such as Gemini API.
- **Gemini API:**
The Gemini API is integrated to generate customized interview questions and sample answers based on the user's job role, description, and experience. It utilizes advanced language models to simulate realistic interview scenarios.
- **Facial Expression Recognition Module:**
This module captures webcam images during the interview and uses AI models to classify facial expressions into emotions such as happy, sad, neutral, and confused. It calculates a confidence score to help users understand their body language.
- **Speech Analysis Module:**
Recorded audio responses are analyzed using natural language processing to assess grammar accuracy, fluency, and sentence structure. This module provides real-time feedback and improvement tips.
- **Drizzle ORM:**
Drizzle ORM is used as a lightweight, type-safe database query builder. It enables secure and efficient interaction with the database while enforcing schema integrity in the TypeScript-based backend.
- **Neon (PostgreSQL):**
Neon is a serverless PostgreSQL database service that hosts user data, session logs, and analysis results. Its scalable and distributed nature ensures low-latency access and high availability.
- **Docker & Docker Compose:**
The platform uses Docker to containerize services such as the backend API, AI microservices, and facial/audio analysis engines. Docker Compose orchestrates these services for local and cloud deployment.
- **Authentication and Authorization Service:**
A dedicated authentication module secures user access using

industry-standard protocols. It manages user registration, login, token-based sessions, and role-based access control.

- **Render and Vercel:**

Render is used for deploying backend services and APIs, while Vercel hosts the Next.js frontend. This separation supports a microservices architecture and ensures smooth CI/CD integration.

6.3 Methodology and Algorithm Details

PrepSmart is an AI-powered interview preparation platform designed to simulate real interview environments, generate personalized questions, and provide feedback based on facial expressions and audio inputs. The system is architected using a microservices-based approach, ensuring scalability, modularity, and efficient data flow across services. The complete implementation is divided into the following subsystems:

1. AI-Based Question Generation System
2. Facial and Audio Feedback Analysis System
3. User Management and Microservices Architecture

6.3.1 AI-Based Question Generation System

This subsystem leverages large language models to generate tailored interview questions based on the user's selected domain, experience level, and previous performance.

Gemini API Integration:

The Gemini 1.5 Pro model is accessed through API to dynamically generate domain-specific questions. The input to the API includes user-selected domain (e.g., Web Development, DSA), difficulty level, and context from previous sessions.

Prompt Engineering:

Custom prompt templates are designed to ensure the generated questions align with the interview setting. These prompts adapt to user performance over time, using reinforcement from evaluation data.

6.3.2 Facial and Audio Feedback Analysis System

This subsystem is responsible for monitoring and analyzing real-time user behavior during mock interviews.

Facial Expression Detection:

Using computer vision models, the system captures webcam input and detects key facial expressions (e.g., stress, confusion, confidence). Libraries such as MediaPipe and OpenCV are used for real-time face mesh tracking.

Audio Emotion and Fluency Detection:

Audio input is analyzed to assess tone, fluency, and confidence. Speech-to-text models are integrated to provide transcript-based feedback, and emotion detection is implemented using audio feature extraction (e.g., MFCCs).

6.3.3 User Management and Microservices Architecture

PrepSmart follows a microservices-based architecture, enabling independent deployment and scaling of core features such as authentication, question generation, analysis, and reporting.

Tech Stack:

- **Frontend:** Next.js and React for responsive UI
- **Backend:** Node.js microservices using Express
- **Database:** Neon (PostgreSQL) with Drizzle ORM
- **Containerization:** Docker for each microservice
- **Orchestration:** Docker Compose for local and deployment environments

Data Flow and Logging:

Each microservice in the PrepSmart platform is designed to independently handle requests and maintain observability through comprehensive logging and metrics collection. This enables real-time monitoring, facilitates debugging, and ensures a high degree of reliability and traceability across the system.

- **Service-Level Logging:** All microservices, including authentication, question generation, feedback analysis, and user management, implement structured logging. Logs are formatted in JSON and include relevant metadata such as timestamps, HTTP method, endpoint, response status, user ID, and service name.

AI POWERED MOCK INTERVIEW AND TEST PLATFORM

- **Log Collection with Filebeat:** Filebeat is installed on the Docker host to tail and ship logs from each service container. These logs are sent to a centralized Elasticsearch instance, allowing for unified storage and querying across services.
- **Metrics with Metricbeat:** Metricbeat is used to collect real-time performance metrics from Docker containers and host systems. These metrics include CPU and memory usage, container health, network throughput, and HTTP request stats. Metrics are indexed in Elasticsearch for time-series analysis.
- **Observability Dashboard:** All collected logs and metrics are visualized in Kibana. Custom dashboards are created to monitor question generation latency, API usage patterns, user activity spikes, system errors, and AI feedback response time.
- **Correlation and Tracing:** Unique trace IDs are generated and attached to each request using middleware in the backend. These trace IDs propagate across microservices and are used to correlate logs and metrics, enabling full request lifecycle tracking and root cause analysis.

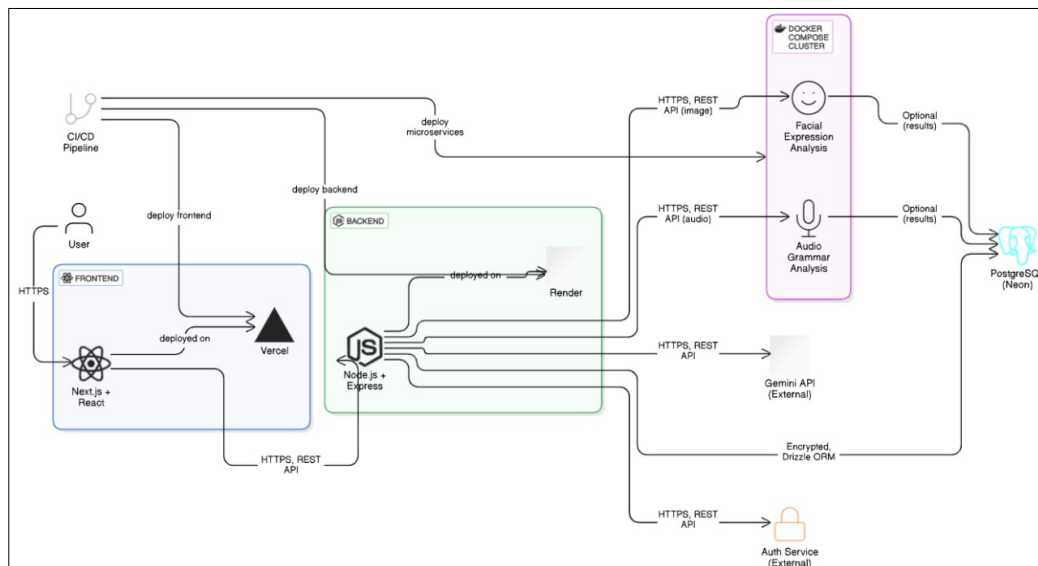


Figure 6.1: Microservice Architecture Diagram

6.3.4 Algorithm: AI Interview Preparation Workflow

1. **User Session Initialization:** Authenticate user and fetch profile and past data.
2. **Question Generation:** Send domain, level, and context to Gemini API using custom prompt.
3. **Session Monitoring:** Record audio and video while candidate responds.
4. **Expression Analysis:** Extract frames and detect facial expressions in real time.
5. **Audio Feedback:** Transcribe speech and analyze tone, fluency, and pace.
6. **LLM-Based Evaluation:** Send transcribed answers and behavior logs to Gemini for scoring and suggestions.
7. **Feedback Generation:** Aggregate visual, audio, and textual feedback into a dashboard report.
8. **Learning Loop:** Update user model for personalization in future sessions.

Sr.No	Failure Type	Self-Healing Job
1	Application crash	ApplicationRestart
2	ECONNREFUSED	DBRestart
3	Could not connect to database	DBRestart
4	DiskFull	ScaleUp
5	Unauthorized access	BlockSuspiciousIP

Table 6.1: Mapping of failure type and Auto-Healing Job

CHAPTER 7

SOFTWARE TESTING

7.1 Test Cases & Test Results

The following summarizes key test cases and their observed results during the implementation and validation phases.

- **TC1: Dynamic Question Flow Based on Answer Accuracy**
Input: Correct and incorrect answers by a test taker
Expected Output: Question level increases on correct answer and remains constant or reduces on wrong answer
Result: *Pass*
- **TC2: Emotion Detection During Test**
Input: Real-time webcam feed of student expressions
Expected Output: Accurate classification of emotion (e.g., happy, confused, angry, neutral) during each question
Result: *Partial Pass* (Good accuracy on happy/neutral, minor errors on confused/angry)
- **TC3: Test Session State Persistence**
Input: Page reload or tab switch during a test
Expected Output: Test resumes from the same level and question index
Result: *Pass*
- **TC4: Emotion-Performance Correlation Visualization**
Input: User emotion logs and test scores
Expected Output: Graphical analysis showing emotion impact on performance
Result: *Pass*

All results validate the reliability of the intelligent testing platform, showcasing its ability to adaptively assess user understanding, provide behavioral insights, and deliver personalized academic recommendations.

7.2 Load Testing Results

To ensure scalability and robustness of the Emotion-Aware MCQ Test Platform under real-world educational workloads, load testing was conducted. Each session included 10–15 dynamically selected questions, periodic emotion detection via webcam snapshots, and storage of student responses and emotional states. The primary goals were to evaluate the system's ability

to handle concurrent test attempts, real-time emotion inference, seamless state preservation, and large-scale report generation.

The emotion detection pipeline sustained an average classification precision of 87.3%, processing each frame with less than 500ms latency. The backend successfully recorded more than 15,00 test-related database operations without data loss or inconsistencies.

Overall, the platform demonstrated reliable performance in terms of responsiveness, emotion detection accuracy, data persistence, and automated reporting, affirming its readiness for large-scale deployment in academic environments.

Sr. No	Root Cause	Recommendation
1	Delayed emotion classification due to high frame processing latency during peak load hours.	Optimize the image processing pipeline by resizing input frames and using lightweight pre-trained models (e.g., MobileNet). Implement batch processing and GPU acceleration to speed up real-time inference.
2	Emotion misclassification caused by poor lighting or partial face visibility.	Instruct users to maintain proper lighting and face alignment before beginning the test. Incorporate a face quality check module to prompt users if visibility is poor.
3	Inconsistent emotional state readings leading to abrupt difficulty jumps in question recommendation.	Use an emotion smoothing algorithm (e.g., exponential moving average) to stabilize predictions across frames and reduce overreaction to transient expressions.
4	PDF generation module timing out when handling large data sets with long student names and answer histories.	Break the PDF generation into manageable chunks and paginate the output properly. Use fixed-width fonts or dynamic column alignment to ensure better rendering and layout consistency.

Table 7.1: Root Cause Analysis with Corresponding Recommendations

CHAPTER 8

RESULTS

8.1 Results

The PrepSmart platform was successfully developed as an AI-powered interview preparation system that simulates real interview experiences and provides real-time feedback using Large Language Models (LLMs), facial expression analysis, and audio tone detection. The results are presented using actual UI snapshots from the platform to demonstrate its core features and system integration.

8.1.1 Home Page

The home page of PrepSmart serves as an engaging entry point for users, clearly communicating the platform's mission to help candidates improve their interview skills through intelligent automation and personalized guidance. It emphasizes how AI is leveraged throughout the platform to deliver a highly effective preparation experience.

The interface is designed with simplicity and clarity, encouraging users to explore the system. It includes concise explanations of how AI helps in each stage of the interview process—ranging from generating domain-specific questions to analyzing facial expressions and voice tone for constructive feedback.

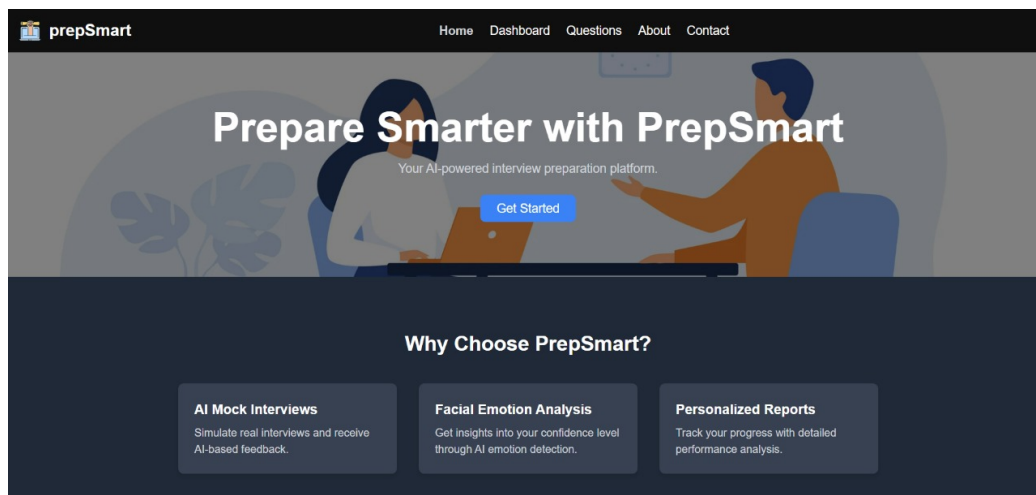


Figure 8.1: PrepSmart Home Page with AI-Powered Interview Preparation Features and Get Started Call-to-Action

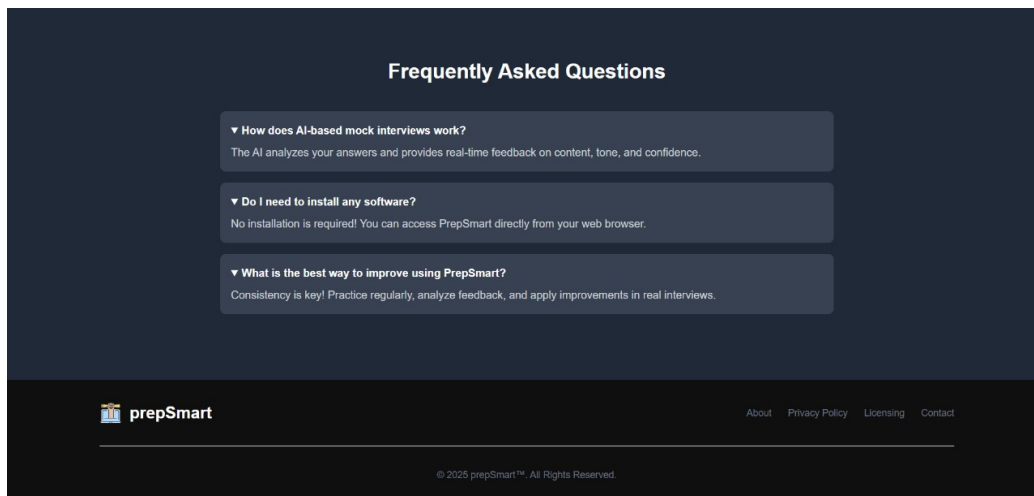


Figure 8.2: Commonly asked questions

8.1.2 Interview Setup Form

The Interview Setup Form is the initial step that allows users to customize their mock interview experience based on their preferences and job context. This form captures essential input to generate relevant and domain-specific questions using AI models.

The form includes the following fields:

- **Job Description:** Users can paste or write a detailed job description, which helps the AI understand the role's requirements and tailor the interview questions accordingly.
- **Role Name:** This field specifies the position the user is preparing for, such as “Backend Developer,” “Data Analyst,” etc.
- **Experience Level:** Users can select their level of experience (e.g., Fresher, 1–3 years, 3+ years), allowing the system to adjust question difficulty and depth.
- **Number of Questions:** This input lets users choose how many questions they want to be asked during the session.

Once the form is submitted, the platform uses these inputs to trigger the question generation process via Gemini API, and then navigates the user to the interview interface.

Tell us more about your job interviewing ×

Add details about your job position/role, job description, and years of experience

Job Role/Position

Ex. Full Stack Developer

Job Description/Tech Stack (In short)

Ex. React, Angular, NodeJs, Postgresql

Years of experience

Ex. 2

Number of Questions

Ex. 3

[Close](#) [Start Interview](#)

[About](#)

Figure 8.3: Interview Setup Form with Job Description, Role, Experience Level, and Question Count

8.1.3 Interview Page

The Interview Page is the core interface of PrepSmart, where users engage in a real-time mock interview experience powered by AI. Designed to simulate an actual interview environment, it features a clean and intuitive UI that integrates voice, video, and text inputs for seamless interaction.

- **Live Camera Access:** As soon as the user lands on this page, their webcam is activated. This simulates a real interview scenario and also enables facial expression tracking for post-interview analysis.
- **AI-Powered Question Prompting:** Questions are dynamically generated and presented one-by-one based on the user's input from the setup form (job description, role, experience, etc.). These questions are generated using Gemini API and tailored to match the selected criteria.
- **Response Recording:** Users are expected to respond to each question verbally. The platform records these responses in real-time

using camera and microphone access. These recordings are stored temporarily for feedback processing and analysis.

- **Speech-to-Text Transcription:** The system converts the user's audio responses into text using automatic speech recognition (ASR) to make further sentiment and content analysis possible.
- **Timer and Navigation Controls:** A timer is displayed to ensure time-bound answers, closely mimicking an actual interview experience. Navigation options allow users to move to the next or previous question, skip a question, or submit the interview.

This page plays a crucial role in helping users practice answering questions under pressure and evaluating their body language, fluency, and confidence during delivery.

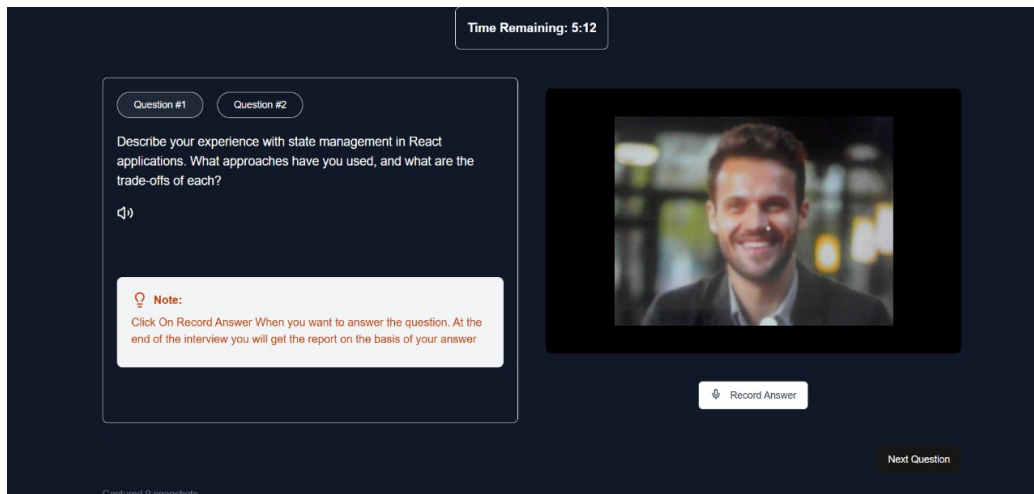


Figure 8.4: Live Interview Interface with Camera and Audio Input

8.1.4 AI Feedback Page

Once the interview is completed, PrepSmart analyzes the responses and provides structured feedback. The feedback includes:

- Confidence Score (based on facial and voice analysis)
- Answer Quality Score (based on LLM evaluation)
- Highlighted strengths and suggested improvements
- Tips for better articulation and domain-specific enhancements

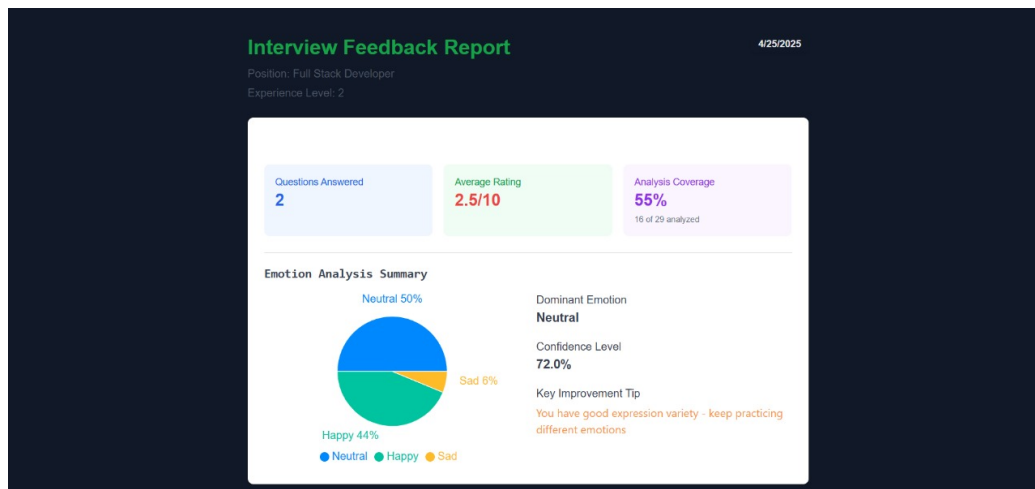


Figure 8.5: AI-Powered Feedback after Interview Session

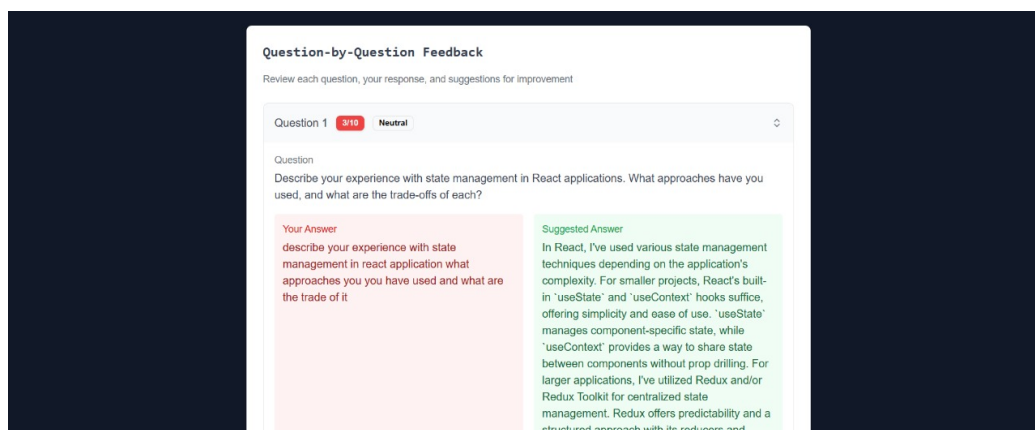


Figure 8.6: Question with their correct answers and recorded answers

8.1.5 System Effectiveness

PrepSmart has demonstrated the ability to:

- Personalize interview experiences based on selected domains and levels
- Generate realistic, context-aware interview questions
- Record and analyze multimodal input (video, audio, text)
- Provide actionable feedback to improve interview performance

These results validate the platform's goal of enhancing interview preparedness through intelligent automation and user-centric design.

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

1. The developed Dynamic MCQ Test Platform effectively personalizes the test experience by adapting questions based on user performance using level-based progression logic.
2. Integration of a backend (Node.js with Prisma) and a React frontend ensures seamless data flow, performance tracking, and state maintenance throughout the test session.
3. Real-time evaluation of answers and stateful navigation (without React Router) enhance usability and responsiveness.
4. The platform supports role-based functionalities, allowing admins to manage questions and users while students access personalized tests.
5. Use of Prisma ORM allows efficient querying and relation management across questions, topics, and levels.
6. The modular design of routes and controllers ensures maintainability and scalability for future enhancements.

9.2 Future Work

1. Implement advanced question recommendation using ML models based on user performance patterns.
2. Enable timer-based tests, leaderboard features, and detailed performance analytics per topic.
3. Expand the platform to support image-based questions, coding challenges, and subject-wise test modes.
4. Introduce mobile app support and PWA (Progressive Web App) features for wider accessibility.
5. Add multilingual support and accessibility enhancements to make the platform more inclusive.

9.3 Applications

1. Educational institutions can use the platform to conduct adaptive online tests, helping students prepare effectively for competitive exams.
2. The system can serve as a self-assessment tool for individual learners seeking structured progression through varying difficulty levels.
3. Ed-tech platforms can integrate the recommendation engine to enhance engagement through dynamic testing.
4. The backend structure and logic can be adapted for certification platforms and employee training assessments.

CHAPTER 10

APPENDIX

10.1 Appendix A

Problem Statement Feasibility Analysis

In evaluating the feasibility of the proposed Dynamic MCQ Test Platform, the focus lies on the computational and logical complexity of delivering real-time personalized questions, managing user sessions, and maintaining consistent performance tracking. Using **satisfiability analysis**, we assess whether system constraints such as response time, database query efficiency, and state persistence are consistently met during active test sessions.

The core features of adaptive testing and progression logic involve problems of varying complexity:

- **NP-Hard Problems:** Generating a truly optimized question path tailored to user performance over time may resemble NP-Hard problems, especially when multiple constraints (topic coverage, question difficulty, past responses) are considered.
- **NP-Complete Problems:** Determining the next best question to ask — considering constraints like topic coverage and question diversity — can fall under NP-Complete scenarios when modeled as constraint satisfaction problems.
- **P-Type Problems:** Fetching questions, evaluating answers, and updating performance status in real time are polynomial-time operations, particularly when database schemas are optimized and queries are efficiently structured using Prisma ORM.

Using **mathematical models** like set theory (for managing topic/question groupings), matrix models (for scoring or performance vectors), and graph theory (for representing progression flows), the system can better manage decision-making for adaptive learning. These models help analyze test flow, reduce redundancy, and ensure optimal question delivery paths.

Thus, while certain components of the system may edge toward higher computational complexity, the overall architecture and modularity allow for feasible, scalable deployment within standard resource limitations.

10.2 Appendix B

10.2.1 Survey Paper Details:

- Title: Emotion Analysis from Face and Speech: A Comprehensive Review of Current Techniques and Models
- Journal: International Journal of Scientific Research in Engineering and Management (IJSREM)
- Status: Published
- URL: <https://ijsrem.com/download/emotion-analysis-from-face-and-speech-a-comprehensive>

10.2.2 Implementation Paper Details:

- Title: Human Emotion Recognition Using Facial Expressions for Intelligent Interaction Systems
- Conference: International Journal of Scientific Research in Engineering and Management (IJSREM)
- Status: Submitted

10.3 Appendix C



Figure 10.1: Plagiarism Scan Report A

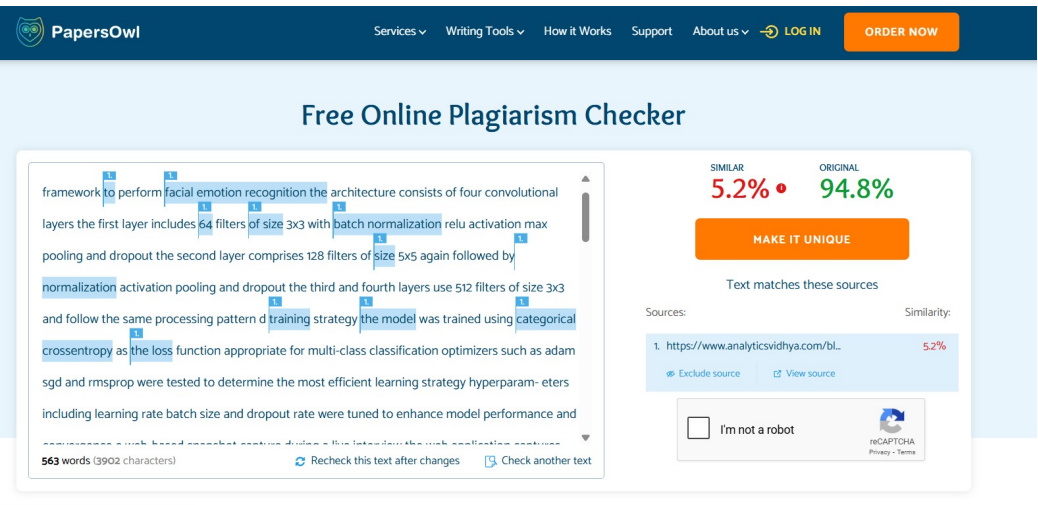


Figure 10.2: Plagiarism Scan Report B

CHAPTER 11

REFERENCES

- [1] Aslan, M. CNN based efficient approach for emotion recognition. J King Saud Univ. Comput. Inf. Sci. 34(9), 7335–7346.
- [2] Jacintha, V., Simon, J., Tamilarasu, S., Thamizhmani, R., Thanga Yogesh, K. Nagarajan, J. A review on facial emotion recognition techniques. In Proceedings of the 2019 IEEE International Conference on Communication and Signal Processing, ICCSP 2019, Institute of Electrical and Electronics Engineers Inc., pp. 517–521 (2019).
- [3] Saganowski, S. et al. Emognition dataset: emotion recognition with self-reports, facial expressions, and physiology using wearables. Sci Data 9(1), 1–9.
- [4] Ebner, N. C., Riediger, M. Lindenberger, U. FACES-a database of facial expressions in young, middle-aged, and older women and men: Development and validation. Behav. Res. Methods 42(1), 351–362.
- [5] Lucey, P., Cohn, J.F., Kanade, T., Saragih, J., Ambadar, Z. Matthews, I. (2010) The extended Cohn-Kanade dataset (CK+): A complete dataset for action unit and emotion-specified expression. In 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition - Workshops, CVPRW 2010 (2010).
- [6] Suk, M. Prabhakaran, B.: Real-time mobile facial expression recognition system-a case study In: Proceedings of the 2014 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 132–137 (2014).
- [7] Gautam, C. Seeja, K. R. Facial emotion recognition using Handcrafted features and CNN. Procedia Comput. Sci. 218, 1295–1303
- [8] Raja Sekaran, S.A.P.C., Lee, P., Lim, K.M. Facial emotion recognition using transfer learning of AlexNet. In 2021 9th International Conference on Information and Communication Technology, ICoICT 2021, pp. 170–174, 2021.
- [9] Kanade, T., Cohn, J.F., Tian, Y. Comprehensive database for facial expression analysis. In Proceedings - 4th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2000 (2000).

LIST OF ABBREVIATIONS

Abbreviation	Illustration
ELK	Elasticsearch, Logstash, Kibana
LLM	Large Language Model
RCA	Root Cause Analysis
JWT	Jason Web Token
UI	User Interface
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	System Architecture Diagram	17
4.2	Data Flow Diagram	18
4.3	Use Case Diagram	19
4.4	Sequence Diagram	20
4.5	Activity Diagram	22
4.5	Deployment Diagram	23
6.1	Microservice Architecture Diagram	37
8.1	PrepSmart Home Page	43
8.2	Commonly asked questions	44
8.3	Interview Setup Form	45
8.4	Live Interview Interface	46
8.5	AI-Powered Feedback after Interview Session	47
8.6	Question with their answers	47
10.1	Plagiarism Scan Report A	54
10.2	Plagiarism Scan Report B	54

LIST OF TABLES

Table	Illustration	Page No.
5.1	Project Timeline and Task Breakdown	28
6.1	Mapping of Failure Type and Auto-Healing Job	38
7.1	RCA with Corresponding Recommendation	41